Contents

Homework 2 —Orbit Integrators: Convergence, Conservation, and Few-Body Dynamics **1**
Overview . . . 1
Problem 1 —Integrator suite behind one interface (20 points) . . . 2
Problem 2 —Kepler convergence study (20 points) . . . 2
Problem 3 —Energy and angular-momentum drift over 10 000 orbits (20 points) . . . 3
Problem 4 —Few-body application (25 points) . . . 4
Problem 5 —AI usage log (10 points) . . . 6
Repository & report quality (5 points) . . . 6
Submission instructions . . . 7
Grading . . . 7
Optional extensions (ungraded; up to +5 bonus) . . . 8
References . . . 8
Appendix A —Reference steppers . . . 8
Appendix B —Direct-summation gravity with Plummer softening . . . 10
Appendix C —Sampling a Plummer sphere in Hénon units . . . 10
Appendix D —The rotating frame for Option A . . . 11

Homework 2 —Orbit Integrators: Convergence, Conservation, and Few-Body Dynamics

Course: 11510ASTR660000 Computational Astrophysics (Fall 2026, NTHU) **Assignment:** HW2 —Orbit Integrators **Released:** Wednesday 2026-10-14 **Due:** Wednesday 2026-10-28, 14:20 (start of the lecture; Taiwan time, UTC+8) **Total: 100 points**

Overview

This assignment turns the two lectures on orbit integration (initial-value problems and Runge–Kutta methods; symplectic integrators and few-body dynamics) into a working, tested integrator suite and uses it on real celestial mechanics. You will put forward Euler, RK2, RK4 and leapfrog behind a single interface, *measure* their convergence orders on the Kepler problem, compare their energy and angular-momentum behavior over 10 000 orbits, and then apply the suite to a few-body system of your choice. The course theme applies throughout: spec the task, let an AI agent implement pieces of it if you wish, and then verify against physics —analytic solutions, conservation laws, exact invariants and measured convergence orders are the arbiters, not the agent's confidence.

The handout is self-contained: every formula, initial condition and reference routine you need is in the problems or in the Appendix at the end. The natural starting point is the `astro-utils` package you built in class (`integrate()` with Euler/RK2/RK4, then `leapfrog()`), but a fresh repository is equally acceptable.

AI tools are **permitted and encouraged on every part** of this assignment, per course policy. Document their use in the AI usage log (Problem 5).

Work in natural units $G = 1$ throughout unless a problem states otherwise. Every tolerance below was measured with a reference implementation of the same setup; they are checked as stated.

Problem 1 —Integrator suite behind one interface (20 points)

Task

Build (or extend) a package with one driver for second-order systems $\ddot{\mathbf{x}} = \mathbf{a}(\mathbf{x})$, e.g.

```
x, v = orbit(acc, x0, v0, t, method="leapfrog")
```

where `acc(x)` returns an array shaped like `x`, `t` is a 1-D array of output times (the n-th step is `t[n+1] - t[n]`), the result arrays have shape `(len(t),) + x0.shape` with row 0 equal to the initial state, and `method` $\in$ {`"euler"`, `"rk2"`, `"rk4"`, `"leapfrog"`}. The Runge–Kutta methods act on the first-order form $\mathbf{y} = (\mathbf{x}, \mathbf{v})$ (wrap a general `integrate(f, y0, t, method)` if you already have one); leapfrog is the kick-drift-kick scheme of Appendix A, applied directly. **Adding a method must not change call sites**, and all four methods must be selectable from the same driver script. Write the spec (`SPEC.md`) before implementation; if an agent implements a method, the acceptance tests below are yours to write and must be committed before the agent runs.

Required tests (committed, `pytest`)

1. **Order, measured.** Harmonic oscillator $\ddot{x} = -x$, $x(0) = 1$, $v(0) = 0$, integrated to $t = 10$ with $h = 0.02$ and $h = 0.01$. The ratio of the errors $|x(10) - \cos 10|$ must be within 20% of 2 (Euler), 4 (RK2 and leapfrog) and 16 (RK4).
2. **Leapfrog's energy error is bounded and predictable.** Same oscillator, $h = 0.01$, $t \in [0, 100]$, energy $E = \frac{1}{2}(v^2 + x^2)$ sampled at every step: $\max |\Delta E/E_0|$ must be within 10% of $h^2/4 = 2.5 \times 10^{-5}$ (for the oscillator the leapfrog energy error is exactly this oscillation; it does not grow).
3. **Time-reversibility.** Kepler orbit with $e = 0.5$ (Problem 3 initial conditions), 100 steps per orbit: integrate one orbit forward, negate the velocity, integrate one more orbit. Leapfrog must return to within 10^{-12} of the initial position. The *same* test applied to RK4 must give a residual above 10^{-6} (RK4 is not reversible); assert that too, so the test cannot pass by accident. Use the Kepler orbit, not the oscillator: on a linear problem RK4 is reversible to 10^{-11} and the test would not discriminate.
4. **One force evaluation per step.** Wrap `acc` in a counter: for N steps leapfrog calls it exactly $N + 1$ times (the closing kick of one step and the opening kick of the next share one evaluation).

Acceptance criteria

- `pytest` passes from a clean clone; all four methods callable through one interface; the tests above included with the stated tolerances.

Problem 2 —Kepler convergence study (20 points)

Task

Two-body Kepler problem reduced to one body, $\ddot{\mathbf{r}} = -\mathbf{r}/r^3$ (i.e. $GM = 1$). Use the circular orbit $r = 1$: $\mathbf{r}_0 = (1, 0)$, $\mathbf{v}_0 = (0, 1)$, period $P = 2\pi$. The orbit returns to $\mathbf{r}_0$ after one period, so the final-state error needs no formula.

1. Integrate **one orbit** with each of the four methods at six step sizes in a geometric sequence, $h = P/100, P/200, \ldots, P/3200$.

2. Define the error as $\|\mathbf{r}(P) - \mathbf{r}_0\|$.
3. Plot error vs h on log-log axes, all methods on one figure, with a least-squares slope for each. Also report the two-point (Richardson) estimate $p \approx \log_2[E(h)/E(h/2)]$ for each adjacent pair.
4. Repeat the runs for **ten orbits** with the same step sizes. One method's errors no longer follow a power law at all, and another's apparent slope exceeds its order. Identify both and explain them with the two ends of a convergence plot: the pre-asymptotic coarse end, where the error is of order one and the orbit is no longer an orbit, and the roundoff floor near 10^{-13}, which more steps only raise.

Required outputs

- `figures/convergence.png` (log-log, one orbit, slopes annotated) + the script.
- Table of fitted slopes and pairwise estimates in the report, for one and ten orbits.

Acceptance criteria

- One-orbit slopes within **0.2** of the theoretical orders: Euler 1, RK2 2, leapfrog 2, RK4 4.
- Fits use ≥ 4 points each; any excluded point (roundoff floor) named and justified.
- The ten-orbit discussion identifies the saturated method and the pre-asymptotic one, with the mechanism.

Problem 3 —Energy and angular-momentum drift over 10 000 orbits (20 points)

Task

Eccentric Kepler orbit: $e = 0.5$, semi-major axis $a = 1$ ($P = 2\pi$, $E_0 = -1/2$, $L_0 = \sqrt{3}/2$). Start at apocenter: $\mathbf{r}_0 = (1 + e,\ 0) = (1.5, 0)$, $\mathbf{v}_0 = (0,\ \sqrt{(1-e)/(1+e)}) = (0, 1/\sqrt{3})$.

1. Run leapfrog and RK4 at **100 steps per orbit for 10 000 orbits**, and again at **400 steps per orbit for 1000 orbits**. Run forward Euler at 400 steps per orbit for 100 orbits (it will not survive longer).
2. Sample $E = \frac{1}{2}v^2 - 1/r$ and $L = xv_y - yv_x$ at **every step** (once per orbit at a fixed phase you can read almost anything on an eccentric orbit). Record, for each orbit, the maximum of $|\Delta E/E_0|$ within the orbit, the value at the end of the orbit, and the maximum of $|\Delta L/L_0|$. Do not store 10^6 states: accumulate the per-orbit numbers inside the step loop.
3. Discuss: why does RK4 (fourth order!) drift secularly while leapfrog's error stays bounded? What does quadrupling the number of steps do to each method, quantitatively? Why can no explicit Runge–Kutta method, at any order, do what leapfrog does here?

Required outputs

- `figures/drift_energy.png`: per-orbit max $|\Delta E/E_0|$ vs orbit number (log-log), all runs overlaid.
- `figures/drift_angmom.png`: per-orbit max $|\Delta L/L_0|$ vs orbit number.
- `figures/euler_orbit.png`: the Euler trajectory (it spirals outward).

Acceptance criteria

- Leapfrog, 100 steps per orbit: max $|\Delta E/E_0| < 2 \times 10^{-2}$ over **all** 10 000 orbits, with no secular trend: the maximum over orbits 9501–10 000 equals the maximum over orbits 1–500 to within

5%.

- Leapfrog, 400 steps per orbit: max $|\Delta E/E_0| < 10^{-3}$ over the 1000 orbits; the ratio of the two leapfrog bounds is 16 within 20% (second order).
- Leapfrog: $|\Delta L/L_0| < 10^{-12}$ throughout (kicks are radial, so L is conserved to roundoff).
- RK4: the end-of-orbit $|\Delta E/E_0|$ grows linearly —a log-log fit against orbit number over orbits 10–1000 has slope 1.0 ± 0.1 at both resolutions —and the ratio of the RK4 errors at orbit 1000 between the two resolutions is $4^5 = 1024$ within 20% (the loss per orbit scales as h^5). Report RK4's $|\Delta L/L_0|$ as well; it is not small.
- Euler: $|\Delta E/E_0|$ exceeds 0.1 within the first orbit; the figure shows the spiral.

Problem 4 —Few-body application (25 points)

Choose **exactly one** option. State your choice and setup in the report. All options must use *your* suite from Problem 1. Appendix B gives a direct- summation acceleration routine for N bodies with optional Plummer softening; Appendix D gives the rotating-frame transformation for Option A; Appendix C gives the Plummer sampler for Option C.

Option A —Sun–Jupiter–Trojan (co-orbital motion)

Three bodies in the inertial barycentric frame, integrated with leapfrog. Units: $G = 1$, total mass 1, separation 1, so Jupiter's period is $P_J = 2\pi$ and the Sun–Jupiter line rotates at $\Omega = 1$. Mass ratio $\mu = m_J/(M_\odot + m_J) = 9.537 \times 10^{-4}$. Initial conditions (all bodies on circular motion about the origin):

body	mass	position	velocity
Sun	$1-\mu$	$(-\mu,\ 0)$	$(0,\ -\mu)$
Jupiter	μ	$(1-\mu,\ 0)$	$(0,\ 1-\mu)$
test particle	0	see below	see below

Three test-particle runs, each ≥ 200 Jupiter orbits at $h \leq P_J/400$:

- (i) **exactly at L4**, the apex of the equilateral triangle: $\mathbf{r} = (\frac{1}{2} - \mu,\ \frac{\sqrt{3}}{2})$, $\mathbf{v} = (-\frac{\sqrt{3}}{2},\ \frac{1}{2} - \mu)$ (rigid rotation, $\mathbf{v} = \Omega \times \mathbf{r}$).
- (ii) on the unit circle **40° ahead of Jupiter**, co-rotating: $\mathbf{r} = (\cos\theta, \sin\theta)$, $\mathbf{v} = (-\sin\theta, \cos\theta)$ with $\theta = 40°$.
- (iii) the same with $\theta = 25°$.

Analysis, in the frame **co-rotating with Jupiter as it actually moves in your run**: rotate every position by $-\varphi_J(t)$, where $\varphi_J = \text{atan2}(y_J, x_J)$ is Jupiter's barycentric angle in the integration —not by $-\Omega t$. (Leapfrog's clock runs slow: at $P_J/400$ Jupiter lags Ωt by about 14° after 480 orbits, and a frame rotated by Ωt shows a spurious drift.) Report the particle's longitude relative to Jupiter, its barycentric radius, its closest approach to Jupiter in Hill radii $r_{\rm H} = (\mu/3)^{1/3} = 0.0683$, and the **Jacobi energy**, the only exact invariant of the problem, computed from inertial-frame quantities as

$$E_J = \tfrac{1}{2}\,(v_x^2 + v_y^2) - \frac{1-\mu}{r_1} - \frac{\mu}{r_2} - (x\,v_y - y\,v_x),$$

with r_1, r_2 the distances to the Sun and to Jupiter *at their current positions*. Plot the three trajectories in the rotating frame (Jupiter fixed at $(1-\mu, 0)$) and the relative longitude against time.

Finally, measure Jupiter's phase lag $\varphi_J(t_{\text{end}}) - \Omega t_{\text{end}}$ for two step sizes (e.g. $P_J/400$ and $P_J/1000$).

Acceptance:

- (i) stays within 10^{-3} of L4 in the rotating frame for the whole run.
- (ii) is a **tadpole** orbit: longitude relative to Jupiter stays inside $(30°, 100°)$, barycentric radius inside $[0.96, 1.04]$, closest approach $> 8\, r_{\text{H}}$.
- (iii) is a **horseshoe** orbit: the longitude passes through 180° (the particle crosses the far side of the Sun and visits the L5 side), it never comes closer than $4\, r_{\text{H}}$ to Jupiter, and it never enters Jupiter's Hill sphere.
- $|\Delta E_J/E_J| < 10^{-5}$ in all three runs; the Sun–Jupiter pair's own energy conserved to $|\Delta E/E_0| < 10^{-6}$.
- The phase lag scales as h^2: the ratio between the two step sizes is $(h_1/h_2)^2$ within 20%.

Option B —The figure-eight and the Pythagorean problem

Equal masses $m_i = 1$, $G = 1$, planar, no softening. The figure-eight choreography (Moore 1993; Chenciner & Montgomery 2000):

$$\mathbf{x}_1 = (0.97000436,\ -0.24308753), \quad \mathbf{x}_2 = -\mathbf{x}_1, \quad \mathbf{x}_3 = (0, 0)$$
$$\mathbf{v}_3 = (-0.93240737,\ -0.86473146), \quad \mathbf{v}_1 = \mathbf{v}_2 = -\tfrac{1}{2}\mathbf{v}_3$$

Period $T = 6.32591398$. Integrate one period with leapfrog and with RK4, both at $h \le 10^{-3}$; plot all three trajectories (they trace a single figure-eight). Then start a twin run with x_1 shifted by 10^{-4} and record $\delta(t) = \big[\sum_i \|\mathbf{x}_i^a - \mathbf{x}_i^b\|^2\big]^{1/2}$ over 20 periods.

The **Pythagorean problem** (Burrau 1913): masses 3, 4, 5 at rest at $(1, 3)$, $(-2, -1)$, $(1, -1)$, $G = 1$. Its close encounters are arbitrarily close, so use Plummer softening $\varepsilon = 0.05$ (Appendix B). Integrate to $t = 40$ with leapfrog at $h = 2.5 \times 10^{-4}$, plus a twin run with one coordinate of one body shifted by 10^{-10}; fit the growth rate λ of $\ln \delta(t)$ over the exponential phase (from $t \approx 2$ until δ reaches 10^{-3}). Repeat at $h = 10^{-4}$ and $h = 5 \times 10^{-4}$.

Acceptance:

- Figure-eight: after one period each body returns to its initial position within 10^{-5} (leapfrog) and 10^{-6} (RK4); $|\Delta E/E_0| \le 10^{-5}$ (leapfrog) and $\le 10^{-10}$ (RK4) along the way.
- Figure-eight twin: δ grows at most linearly, $\delta(20T)/\delta(T) < 50$ (it is a phase drift, not an instability: the figure-eight is linearly stable, Simó 2002). State this contrast with the next item explicitly.
- Pythagorean: λ from the $h = 2.5 \times 10^{-4}$ and 10^{-4} runs agree within 20% and give a Lyapunov time $t_\lambda = 1/\lambda \in [1.2, 2.0]$; max $|\Delta E/E_0| < 2 \times 10^{-2}$ at $h = 2.5 \times 10^{-4}$ (the transient at the closest encounter), smaller at 10^{-4}. The 5×10^{-4} run gives a different λ and a larger energy transient: explain why it is not a valid measurement.
- Figures: `figures/figure_eight.png`, `figures/lyapunov.png` ($\log \delta$ vs t for the three step sizes).

Option C —Small N-body: Plummer sphere

$N = 100$ equal-mass particles sampled from a Plummer sphere in Hénon (N-body) units: $G = 1$, $M = 1$, $E = -1/4$, which fixes the Plummer scale radius $a = 3\pi/16 = 0.5890$, the half-mass radius $r_h = 1.305\,a = 0.769$, $K = 1/4$, $W = -1/2$ (unsoftened) and the crossing time $t_{\text{cross}} = 2\sqrt{2} = 2.83$. The sampler is in Appendix C: radii by inverting the cumulative mass $M(r)/M = r^3/(r^2 + a^2)^{3/2}$, directions isotropic, speeds by rejection from $p(q) \propto q^2(1 - q^2)^{7/2}$ with $q = v/v_{\text{esc}}(r)$ and $v_{\text{esc}} = \sqrt{2}\,(r^2 + a^2)^{-1/4}$. Direct $\mathcal{O}(N^2)$ summation with Plummer softening $\varepsilon = 0.05$; leapfrog with fixed $h \leq 10^{-3}$; integrate to $t = 10$ (about 3.5 crossing times). Every $\Delta t = 0.1$ record the softened total energy, the virial ratio $-2K/W$, and the Lagrangian radii enclosing 10%, 50% and 90% of the mass about the center of mass.

- Verify the sampler before evolving anything: $-2K/W$ and r_h of the sample against the theory values (sampling noise is $\sim N^{-1/2}$).
- Verify the force routine: for one snapshot, your vectorized accelerations must agree with an explicit double loop to 10^{-12}.
- Plot: $|\Delta E/E_0|$ vs time; $-2K/W$ vs time; the three Lagrangian radii vs time.

Acceptance: initial sample has $-2K/W \in [0.9, 1.1]$ and r_h within 15% of 0.769; $|\Delta E/E_0| < 10^{-4}$ throughout the run (report the number; it should be far below this); time-averaged $-2K/W$ over $t \in [2, 10]$ within $[0.9, 1.1]$; the 50% Lagrangian radius stays within a factor 1.3 of its initial value (no collapse, no explosion; the 90% radius may grow as a few particles evaporate).

Required outputs (all options)

- Setup description (units, initial conditions, h, duration) precise enough to reproduce; the run script committed; figures scripted.

Problem 5 —AI usage log (10 points)

Maintain `AI_LOG.md` covering the whole assignment: tools/models used and for what; key prompts and specs (verbatim or as links to committed transcripts); **at least one case where the AI was wrong and how you caught it** (e.g. an agent-written leapfrog that was actually symplectic Euler, asymmetric half kicks that only the reversibility test sees, a sign error in a Coriolis-type term, a convergence test that silently compared against the wrong analytic solution). If you claim the AI made no mistakes, include the verification evidence that establishes this, item by item.

Acceptance: specific and reproducible; ties each AI contribution to the test that validated it.

Repository & report quality (5 points)

Clean clone runs, pinned environment, README with exact commands, scripted figures, report ≤ 6 pages. Details below.

Submission instructions

1. **GitHub repository** (public, or private with the instructor and TA added as collaborators: GitHub accounts `kuochuanpan` and `liting05`). Post the URL on the HW2 assignment in Google Classroom; you can post it early and keep working until the deadline.
2. **Deadline check = your last commit.** Your submission time is the timestamp of the most recent commit on `main` (author and committer dates both count). If it is after Wednesday 2026-10-28, 14:20 (Taiwan time, UTC+8), the late penalty applies to the whole assignment: 20 points per day late (any part of a day counts as a day), up to 5 days; work more than 5 days late receives no credit. Stop pushing to `main` once you are done. Check with `git log -1 --format='%aI  %cI %s' main`.
3. **Short report** (`report.pdf` or `report.md`, ≤ 6 **pages** with figures) at the repo root: setup, results, the required tables/figures, and the discussion items above.
4. **Clean-clone rule:** `README.md` gives exact commands to install the pinned environment (`requirements.txt` / `environment.yml` / `uv.lock`; Python $\geq$ 3.11, `numpy`/`matplotlib`/`scipy`/`pytest`) and to regenerate every figure. Long runs may cache results to committed `.npz` files, but the generating scripts must be present and documented.
5. Every report figure is produced by a committed script.

Grading

Letter grade per the syllabus (completeness, accuracy, creativity); points map approximately as A ≥ 90, B ≥ 75, C ≥ 60.

Component	Points	Full credit looks like
P1 Integrator suite	20	One interface, four methods, all required tests pass
P2 Convergence study	20	Slopes within 0.2 of 1/2/2/4; clean log-log figure; ten-orbit ends explained
P3 Drift study	20	Tolerances met; bounded-vs-secular behavior shown, scaled and explained
P4 Few-body application	25	Option's acceptance criteria met; reproducible setup
P5 AI usage log	10	Specific, reproducible, includes a caught AI error
Repo & report	5	Clean clone runs; pinned env; ≤ 6 pages; scripted figures

- **Accuracy** hinges on the quantitative criteria: measured orders, conservation tolerances, invariants and stability boxes are checked as stated.
- **Creativity:** e.g. an elegant shared interface, an extra integrator, a sharper diagnostic (libration-angle time series, a Poincaré-style section, a zero-velocity curve drawn under the Trojan orbit).

Optional extensions (ungraded; up to +5 bonus)

- Give leapfrog a state-dependent step, $h_n = 0.05\, r_n^{3/2}$ (a fixed fraction of the local free-fall time), on the $e = 0.5$ orbit of Problem 3, and compare with a fixed step using the **same total number of steps**. Show that the adaptive version loses the bounded energy error —demonstrate, don't just assert —and explain which property of the fixed-step scheme the state-dependent step breaks.
- Option A: scan the start angle θ between 20° and 60° and map the orbit type (tadpole, horseshoe, encounter) against $E_J - \Phi_{\text{eff}}(L_3)$; locate the tadpole/horseshoe boundary to within 1°. Then start at $\theta = 20°$ and show what the close encounter does to E_J at $P_J/400$ versus $P_J/1000$: the closest encounter sets the step.
- Option C: repeat with $N = 400$ and measure the $\mathcal{O}(N^2)$ wall-clock scaling; optionally accelerate the force loop (NumPy vectorization or numba) and verify the optimized forces match the reference to 10^{-12}.
- Reproduce Problem 3 with a symplectic fourth-order (Yoshida) composition of leapfrog and compare the drift.

References

- E. Hairer, C. Lubich, G. Wanner, *Geometric Numerical Integration*, 2nd ed. (Springer, 2006): symplectic and reversible methods, backward error analysis, variable steps.
- P. Bodenheimer et al., *Numerical Methods in Astrophysics* (CRC, 2007): ODE integrators and N-body chapters.
- W. H. Press et al., *Numerical Recipes*, 3rd ed. (Cambridge, 2007): Runge–Kutta and adaptive stepping.
- M. T. Heath, *Scientific Computing*, 2nd ed. (SIAM, 2018): ODE error and stability analysis.
- J. Binney, S. Tremaine, *Galactic Dynamics*, 2nd ed. (Princeton, 2008): chapter 3 (restricted three-body problem, Jacobi constant, Lagrange points), chapter 4 (the Plummer model).
- S. J. Aarseth, *Gravitational N-Body Simulations* (Cambridge, 2003): Plummer sampling, softening, close encounters.
- A. Chenciner, R. Montgomery, *Annals of Mathematics* 152 (2000) 881; C. Moore, *Physical Review Letters* 70 (1993) 3675: the figure-eight. C. Simó, *Contemporary Mathematics* 292 (2002) 209: its linear stability.
- D. C. Heggie, R. Mathieu, in *The Use of Supercomputers in Stellar Dynamics* (Springer, 1986): the standard N-body units.

Appendix A —Reference steppers

All steppers below are pure functions: no hidden state, no printing. `t` is a 1-D array of output times and the n-th step is `h = t[n+1] - t[n]`.

```
import numpy as np

def rk4_step(f, t, y, h):
    """Classical fourth-order Runge-Kutta for dy/dt = f(t, y)."""
    k1 = f(t, y)
    k2 = f(t + 0.5 * h, y + 0.5 * h * k1)
```

```
    k3 = f(t + 0.5 * h, y + 0.5 * h * k2)
    k4 = f(t + h, y + h * k3)
    return y + (h / 6.0) * (k1 + 2.0 * k2 + 2.0 * k3 + k4)

def kdk_step(acc, x, v, a, h):
    """One kick-drift-kick leapfrog step; a = acc(x) is carried between steps.

    Returns (x_new, v_new, a_new). Exactly one call of acc per step.
    """
    vh = v + 0.5 * h * a                # kick, half step
    x_new = x + h * vh                  # drift, full step
    a_new = acc(x_new)                  # the one force evaluation
    v_new = vh + 0.5 * h * a_new        # kick, half step
    return x_new, v_new, a_new

def leapfrog(acc, x0, v0, t):
    """Kick-drift-kick leapfrog (velocity Verlet) for x'' = acc(x).

    Second order, symplectic, time-reversible, one acc() call per step.
    Returns (x, v), each of shape (len(t),) + x0.shape.
    """
    x = np.empty((len(t),) + np.shape(x0), dtype=float)
    v = np.empty_like(x)
    x[0], v[0] = x0, v0
    a = acc(x[0])
    for n in range(len(t) - 1):
        x[n + 1], v[n + 1], a = kdk_step(acc, x[n], v[n], a, t[n + 1] - t[n])
    return x, v
```

Forward Euler is `y + h * f(t, y)`; RK2 (explicit midpoint) is `y + h * f(t + h/2, y + h/2 * f(t, y))`. To run a Runge–Kutta method on a second-order system, stack the state as `y = concatenate([x.ravel(), v.ravel()])` and let `f(t, y)` return `concatenate([v.ravel(), acc(x).ravel()])`.

The Kepler problem for Problems 2–3, $GM = 1$, state $\mathbf{x} = (x, y)$:

```
def kepler_acc(x):
    r3 = (x[0] ** 2 + x[1] ** 2) ** 1.5
    return -x / r3

def energy(x, v):       return 0.5 * np.sum(v * v, axis=-1) - 1.0 / np.hypot(x[..., 0],
↪   x[..., 1])
def ang_mom(x, v):      return x[..., 0] * v[..., 1] - x[..., 1] * v[..., 0]
```

For a bound orbit with semi-major axis a and eccentricity e, $E = -1/(2a)$, $L = \sqrt{a(1-e^2)}$, $P = 2\pi a^{3/2}$, and the speed at radius r follows from vis-viva, $v^2 = 2/r - 1/a$.

Appendix B —Direct-summation gravity with Plummer softening

Positions `x` and velocities `v` have shape `(N, d)`, masses `m` shape `(N,)`, $G = 1$. `eps = 0` is exact point-mass gravity; otherwise every pair interacts through $r^2 \to r^2 + \varepsilon^2$.

```
def nbody_acc(x, m, eps=0.0):
    """Acceleration of every body from every other body; one O(N^2) sweep."""
    dx = x[None, :, :] - x[:, None, :]            # dx[i, j] = x[j] - x[i]
    r2 = np.sum(dx * dx, axis=-1) + eps * eps     # (N, N)
    np.fill_diagonal(r2, np.inf)                  # no self-force
    return np.sum(dx * m[None, :, None] * r2[:, :, None] ** -1.5, axis=1)

def nbody_energy(x, v, m, eps=0.0):
    """Kinetic and potential energy; for eps > 0 the Plummer-softened potential."""
    K = 0.5 * np.sum(m * np.sum(v * v, axis=-1))
    dx = x[None, :, :] - x[:, None, :]
    r2 = np.sum(dx * dx, axis=-1) + eps * eps
    np.fill_diagonal(r2, np.inf)
    W = -0.5 * np.sum(m[:, None] * m[None, :] / np.sqrt(r2))
    return K, W
```

A massless test particle (Option A) is simply a body with `m = 0`: it feels the others and exerts nothing. The softened energy $K+W$ is the quantity leapfrog conserves (approximately) when `eps > 0`; compare against it, not against the unsoftened one.

Appendix C —Sampling a Plummer sphere in Hénon units

```
def sample_plummer(N, a=3 * np.pi / 16, seed=None):
    """Isotropic Plummer sphere, G = M = 1, E = -1/4 in expectation.

    Returns (mass (N,), pos (N, 3), vel (N, 3)), center of mass at rest.
    """
    rng = np.random.default_rng(seed)

    def isotropic(n):                        # random unit vectors, shape (n, 3)
        cth = 2.0 * rng.random(n) - 1.0
        phi = 2.0 * np.pi * rng.random(n)
        sth = np.sqrt(1.0 - cth * cth)
        return np.column_stack([sth * np.cos(phi), sth * np.sin(phi), cth])

    # positions: invert the cumulative mass profile M(r)/M = r^3 / (r^2 + a^2)^(3/2)
    X = rng.random(N)
    r = a / np.sqrt(X ** (-2.0 / 3.0) - 1.0)
    pos = r[:, None] * isotropic(N)

    # speeds: q = v / v_esc with p(q) ~ q^2 (1 - q^2)^(7/2), by rejection (max 0.092 <
    ↪  0.1)
    q = np.empty(N)
```

```
    filled = 0
    while filled < N:
        qt = rng.random(2 * (N - filled) + 16)
        yt = 0.1 * rng.random(qt.size)
        ok = qt[yt < qt ** 2 * (1.0 - qt ** 2) ** 3.5]
        take = ok[: N - filled]
        q[filled: filled + take.size] = take
        filled += take.size
    v_esc = np.sqrt(2.0) * (r ** 2 + a ** 2) ** -0.25
    vel = (q * v_esc)[:, None] * isotropic(N)

    mass = np.full(N, 1.0 / N)
    pos -= np.average(pos, weights=mass, axis=0)   # exact center of mass at rest
    vel -= np.average(vel, weights=mass, axis=0)
    return mass, pos, vel
```

With $N = 100$ the sample's $-2K/W$ and r_h scatter by a few percent around the theory values. If you prefer exact virial numbers, rescale once with $\varepsilon = 0$: multiply the velocities by $\sqrt{-W/(2K)}$, then set $\lambda = -2W$ and map $\mathbf{x} \to \lambda\mathbf{x}$, $\mathbf{v} \to \mathbf{v}/\sqrt{\lambda}$, which gives $W = -1/2$ and $K = 1/4$ exactly. Document whether you did.

Appendix D —The rotating frame for Option A

Given the integrated barycentric positions, with Jupiter at index 1 and the test particle at index 2:

```
phiJ = np.arctan2(x[:, 1, 1], x[:, 1, 0])          # Jupiter's actual angle, per output
 ↪  time
c, s = np.cos(phiJ), np.sin(phiJ)
xr = c * x[:, 2, 0] + s * x[:, 2, 1]               # particle, rotating frame: Jupiter
 ↪  on +x
yr = -s * x[:, 2, 0] + c * x[:, 2, 1]
lon = np.degrees(np.arctan2(yr, xr))               # longitude ahead of Jupiter
```

In this frame the Sun sits at $(-\mu, 0)$, Jupiter at $(1-\mu, 0)$, and the Lagrange points are fixed: $L_4 = (\frac{1}{2} - \mu, \frac{\sqrt{3}}{2})$, $L_5 = (\frac{1}{2} - \mu, -\frac{\sqrt{3}}{2})$, and on the axis $L_1 = 0.93237$, $L_2 = 1.06883$, $L_3 = -1.000397$ for $\mu = 9.537 \times 10^{-4}$ (roots of $x - (1-\mu)(x+\mu)/|x+\mu|^3 - \mu(x-1+\mu)/|x-1+\mu|^3 = 0$). The effective potential, if you want to draw the zero-velocity curve $\Phi_{\text{eff}} = E_J$ under the orbit, is $\Phi_{\text{eff}}(x, y) = -(1-\mu)/r_1 - \mu/r_2 - (x^2 + y^2)/2$.